Dear Students,

The given 5 questions are to be practised and written in a lab report. You are also required to complete the task below and write it in the lab report.

Q-6

Use Java collection to create a min-priority queue of the details of students using their mark as the key {note: the lower the key, the lower the priority}. Assume that the objects are added to the priority queue one after another. Display the list.

Input: Student list with their details {name, roll, mark}: [{"Rabi", 125, 45}, {"Ashok", 111, 42}, {"Zeny", 95, 37}, {"Vicky", 120, 44}]

Output: [{"Zeny", 95, 37}, {"Vicky", 120, 44}, {"Ashok", 111, 42}, {"Rabi", 125, 45}]

	ITER, SIKSHA 'O' ANUSANDHAN (Deemed to be University)		**EXPERIMENT 02**
Branch	**CSE, CS&IT, Data Science, AI&ML, IoT, Cyber Security**	Programme	**B.Tech**
Course Name	**Algorithm Analysis and Design - 2**	Semester	**4th**
Course Code	**CSE 2632**	Academic Year	**2025-2026**
Learning Level (LL)	**L1**: Remembering	**L3:** Application	**L5**: Evaluation
	L2: Understanding	**L4**: Analysis	**L6**: Creation
Each Question Carries 5 Marks			
Objective of Experiment: Use of Java Collection in various data structure.			

Q's	Questions	COs	LL

1	**# ArrayList implementation in JAVA Collections**
	import java.util.ArrayList;
	
	public class ArrayListDemo {
	public static void main(String[] args) {
	ArrayList<Integer> al = new ArrayList<Integer>();
	
	al.add(1); // add 1 to the end of the list
	al.add(2); // add 2 to the end of the list
	System.out.println("Contents of Array: " + al);
	System.out.println("Array Size: " + al.size());
	System.out.println("Array IsEmpty: " + al.isEmpty());
	al.remove(al.size() -1); // last element of array is removed.
	al.removeAll(al); // all the elements of array are removed.
	System.out.println("Array IsEmpty: " + al.isEmpty());
	}
	}
	
	Output:
	Contents of Array: [1, 2]
	Array Size: 2
	Array IsEmpty: false
	Array IsEmpty: true		

2	**#LinkedList implementation in JAVA Collections**
	import java.util.LinkedList;
	public class LinkedListDemo {
	public static void main(String[] args) {
	LinkedList<Integer> ll = new LinkedList<Integer>();
	ll.addFirst(2); // 8 is added to the list
	ll.addLast(10); // 9 is added to last of the list.
	ll.addFirst(1); // 7 is added to first of the list.
	ll.addLast(11); // 20 is added to last of the list
	System.out.println("Contents of Linked List: " + ll);
	ll.removeFirst();
	ll.removeLast();
	System.out.println("Contents of Linked List: " + ll);
			L3

	} } Output: Contents of Linked List: [1, 2, 10, 11] Contents of Linked List: [2, 10]		

3	**#Stack implementation in JAVA Collection**
	public class StackDemo {
	public static void main(String[] args) {
	Stack<Integer> stack = new Stack<Integer>();
	int temp;
	stack.push(1);
	stack.push(2);
	stack.push(3);
	System.out.println("Stack : "+stack);
	System.out.println("Stack size : "+stack.size());
	System.out.println("Stack pop : "+stack.pop());
	System.out.println("Stack top : "+stack.peek());
	System.out.println("Stack isEmpty : "+stack.isEmpty());
	}
	}
	
	Output:
	Stack : [1, 2, 3]
	Stack size : 3
	Stack pop : 3
	Stack top : 2
	Stack isEmpty : false
			L3

4	**#Queue implementation in JAVA Collection**
	import java.util.ArrayDeque;
	
	public class QueueDemo {
	
	public static void main(String[] args) {
	ArrayDeque<Integer> que = new ArrayDeque<Integer>();
	que.add(1);
	que.add(2);
	que.add(3);
	System.out.println("Queue : "+que);
	System.out.println("Queue size : "+que.size());
	System.out.println("Queue peek : "+que.peek());
	System.out.println("Queue remove : "+que.remove());
	System.out.println("Queue isEmpty : "+que.isEmpty());
	}
	}
	
	Output:
	Queue : [1, 2, 3]
	Queue size : 3
	Queue peek : 1
	Queue remove : 1
	Queue isEmpty : false
			L3

5	**#PriorityQueue implementation in JAVA Collection**
	import java.util.PriorityQueue;
	
	public class PriorityQueueDemo {
	public static void main(String[] args) {
	PriorityQueue<Integer> pq = new PriorityQueue<Integer>();
	int[] arr = {1,2,10,8,7,3,4,6,5,9};
	for(int i: arr){
	pq.add(i);
	}
	System.out.println("Printing Priority Queue Heap : " + pq);
	
	System.out.print("remove elements of Priority Queue ::");
	while(pq.isEmpty() == false){
	System.out.print(" " + pq.remove());
	}
	}
	}
	Output
	Printing Priority Queue Heap : [1, 2, 3, 5, 7, 10, 4, 8, 6, 9]
	Dequeue elements of Priority Queue :: 1 2 3 4 5 6 7 8 9 10		

Course Outcomes	CO1	Understand the network flow problem and apply it to real-world problems
	CO2	- Distinguish between computationally tractable and intractable problems. - define and relate class-P, class-NP and class NP-complete, PSPACE, and PSPACE-complete. - Given a problem in NP, define an appropriate certificate and the verification algorithm.
	CO3	Understand approximation algorithms and apply this concept to solve problems.
	CO4	Understand local search techniques and apply this concept to solve problems.
	CO5	Understand randomization and apply this concept to solve problems.
	CO6	Identify and apply an appropriate algorithmic approach to solve a problem and explain the challenges to solve it.